

Теория параллелизма: Задание 2
“Анализ эффективности OpenMP-версии”
“2.1”

Характеристика аппаратной составляющей устройства:

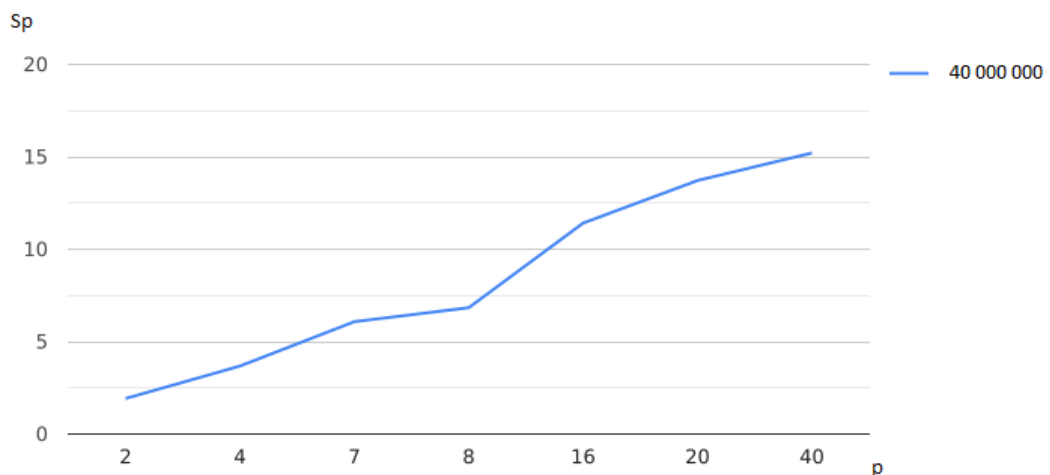
- Процессор (CPU) - Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz, потоков на ядро - 2, ядер на сокет - 20, число сокет - 2, число NUMA-узлов - 2.
- Наименование сервера: ProLiant XL270d Gen10.
- NUMA-узлы - 2: 1) с 0 по 19 и с 40 по 59 потоков.
2) с 20 по 39 и с 60 по 79 потоков.
- Операционная система: Ubuntu 22.04.3 LTS – Ubuntu, семейство - Linux.

В данном задании требуется реализовать многопоточную версию программы численного интегрирования методом средних прямоугольников с использованием OpenMP (см. Репозиторий README.md). Анализ масштабируемости кода приведен ниже в виде таблицы:

M = N	Количество потоков								
	2			4		7		8	
	T1	T2	S2	T4	S4	T7	S7	T8	S8
40 000 000	0.489158	0.254116	1.924939791	0.132858	3.681810655	0.0804369	6.081263699	0.0715411	6.837440297

M = N	Количество потоков					
	16		20		40	
	T16	S16	T20	S20	T40	S40
40 000 000	0,0428441	11,41716129	0,035655	13,71919787	0,032161	15,20966388

Диаграмма масштабируемости:



Где Sp - коэффициент ускорения программы; p - число потоков;
 T_p - время, затраченное на работы программы при “ p ” потоков.

Вывод: при увеличении числа потоков программа начинает фактически линейно ускоряться (если оценивать по коэффициенту Sp). Также при сильно больших значениях количества потоков программа начинает постепенно замедляться в связи с большей тратой времени на передачу данных между NUMA-узлами. Использование атомарных операций, в отличие от критических секций, дают больший прирост в скорости, так как блокировка идет на аппаратном уровне инструкций процессора, а не на уровне блоков кода.